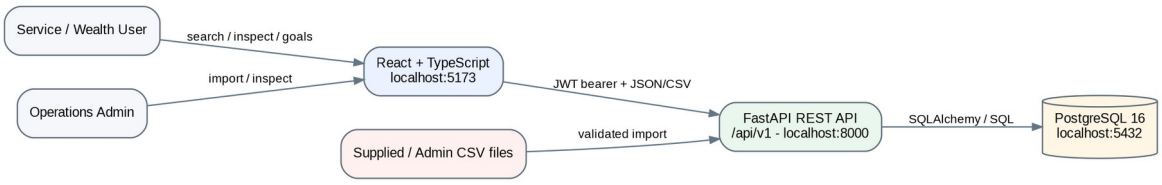
FINPILOT

Architecture & Design Report

Full-Stack Fintech Coding Assignment



Implementation: React + TypeScript + FastAPI + PostgreSQL + Docker Compose

Synthetic assessment data only. Local deployment.

1. Executive overview

FinPilot is a production-minded internal wealth-operations dashboard for inspecting a customer's consolidated investment position, recent activity, risk profile and financial goals. The implementation also provides an ADMIN-only transaction CSV import path with explicit row-level validation and rejection reasons.

The design is intentionally a modular monolith. The supplied dataset and review timebox do not justify microservices. A single FastAPI service keeps authentication, domain validation, imports and database access understandable enough to trace live, while PostgreSQL remains the system of record and performs the important aggregation work.

Layer	Chosen technology	Reason
Frontend	React 19 + TypeScript + Vite	Small client surface, typed UI, fast local development.
API	FastAPI + Pydantic	Explicit request validation, OpenAPI/Swagger and readable route contracts.
Data access	SQLAlchemy 2	Parameterized ORM/query paths while still allowing targeted SQL.
Database	PostgreSQL 16	Required relational database; constraints, views, indexes and SQL analysis.
Migrations	Alembic	Reproducible schema creation from a clean database.
Runtime	Docker Compose	One-command local environment for reviewer reproducibility.
CI	GitHub Actions	Backend lint/tests and frontend production build on push/PR.

Major design decisions: aggregate portfolio values in PostgreSQL, preserve raw CSV files unchanged, reject invalid import rows rather than silently correcting them, use seeded local roles for the assessment and document production-hardening gaps explicitly.

2. System context

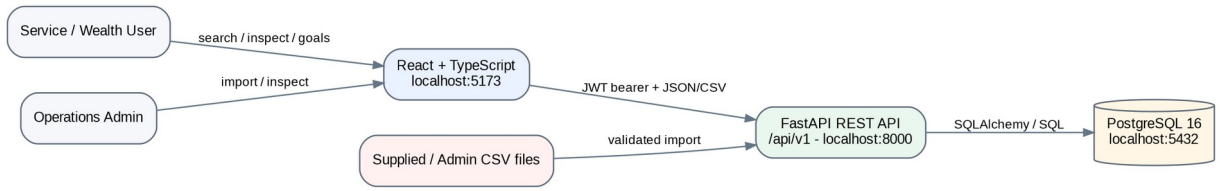


Figure 1. FinPilot system context.

The two local personas are a VIEWER and an ADMIN. Both use the React application. Protected API endpoints require a short-lived bearer token. ADMIN additionally receives access to the transaction import workflow. No external identity provider or market-data provider is required because the assignment uses synthetic data.

3. Component architecture

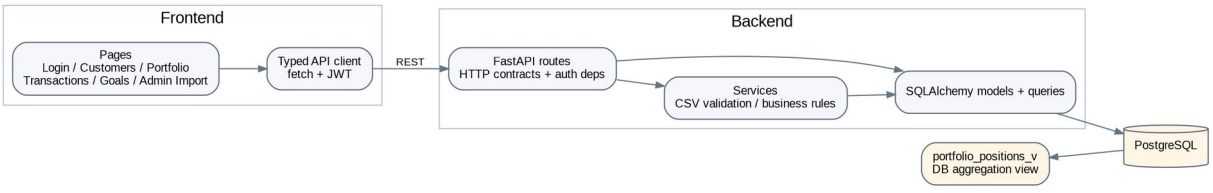


Figure 2. Component architecture and responsibility boundaries.

Frontend pages are split by user task rather than placed into a single dashboard component. A shared API client attaches the JWT and converts non-2xx responses into UI errors. On the backend, FastAPI routes own HTTP semantics and authorization; the CSV import rules live in a service that can be tested without HTTP; SQLAlchemy models encode relational constraints; and the portfolio view centralizes valuation calculations.

4. ERD and data model

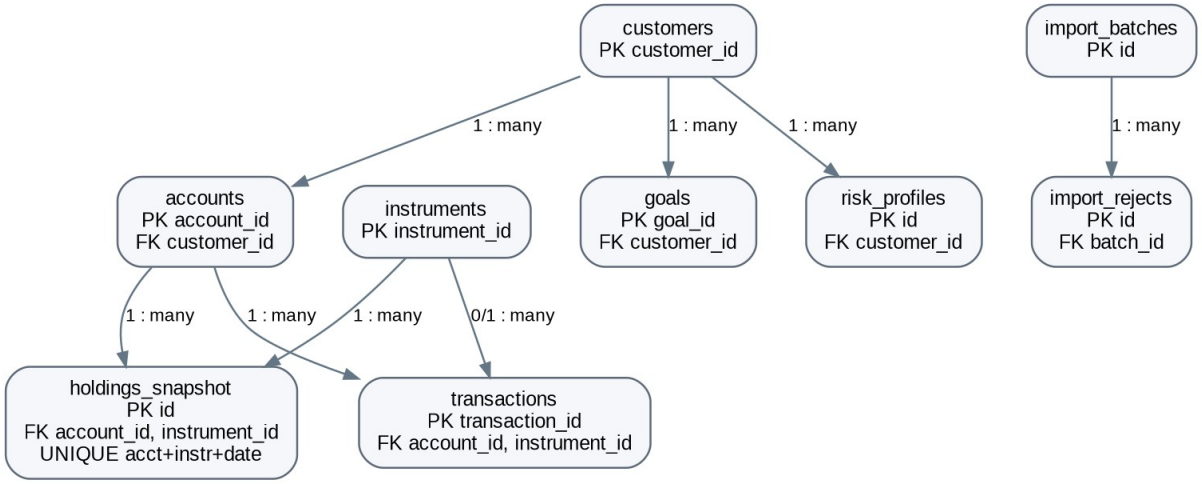


Figure 3. Core relational model.

Table	Key controls / query purpose
customers	Primary customer ID; unique email; indexed name/email/city; KYC and segment checks.
accounts	FK to customer; indexed customer_id; account type/status checks.
instruments	Primary instrument ID; unique symbol; price and asset-class attributes.
holdings_snapshot	FKs to account/instrument; unique account+instrument+snapshot date; non-negative quantity/cost; composite account/date index.
transactions	Primary transaction ID; account/instrument FKs; type/status checks; account+trade_date index for filtered pagination.
goals	FK to customer; positive target and non-negative funded amount; priority check.
risk_profiles	FK to customer; unique customer+assessed date; score/horizon/liquidity checks.
import_batches / import_rejects	Audit summary for each import and row-level rejection reasons.

4.1 Derived portfolio view

`portfolio_positions_v` joins holdings, accounts and instruments and calculates market value and unrealised P/L in PostgreSQL. The API then groups those valued rows into customer totals, account totals and asset-class allocation. This avoids loading raw holdings into browser JavaScript for core portfolio calculations.

4.2 Data-quality handling

Anomaly in supplied data	Observed example	Chosen handling
Duplicate transaction ID	T0000026 appears twice	First accepted row wins; duplicate is rejected and recorded.
Invalid transaction instrument	T0004551 -> I9999	Rejected because instrument FK is unknown.

Negative amount	T0004552 FEE amount -75	Rejected under the chosen unsigned-amount source contract.
Future trade date	T0004553 -> 2027-01-05	Rejected as a future trade date.
Duplicate holdings	Three duplicated account/instrument/snapshot keys	Loader skips duplicate raw positions; DB unique constraint prevents persistence duplication.

The raw CSV files remain unchanged. Validation decisions are explicit and reproducible. For a real integration, the signed/unsigned meaning of FEE amounts would be agreed in a source data contract before changing the rule.

5. API design

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Authenticate seeded reviewer user and return JWT + role.
GET	/api/v1/customers	Search/filter/paginate customers.
GET	/api/v1/customers/{id}	Profile, latest risk summary and accounts.
GET	/api/v1/customers/{id}/portfolio	Customer/account totals, allocation and valued positions.
GET	/api/v1/customers/{id}/transactions	Server-side date/account/instrument/type filters and pagination.
GET/POST	/api/v1/customers/{id}/goals	List goals / create a validated goal.
PATCH	/api/v1/goals/{goal_id}	Edit allowed goal fields.
POST	/api/v1/admin/imports/transactions	ADMIN-only validated CSV import with rejection details.
GET	/health	API/database health without exposing secrets.

FastAPI publishes the machine-readable OpenAPI definition and interactive Swagger UI at `/docs`. The API is versioned under `/api/v1`. Authentication and ADMIN role checks are enforced in server dependencies rather than only in the browser.

5.1 Pagination and errors

Customer and transaction lists use backend pagination with bounded page sizes. Transaction filters are translated into SQL predicates. Not-found conditions return 404 responses and validation failures use FastAPI/Pydantic error semantics or concise domain `detail` messages.

6. Workflow diagrams



Figure 4. Login workflow.



Figure 5. Customer portfolio read workflow.

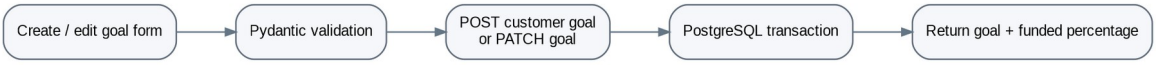


Figure 6. Goal create/edit workflow.

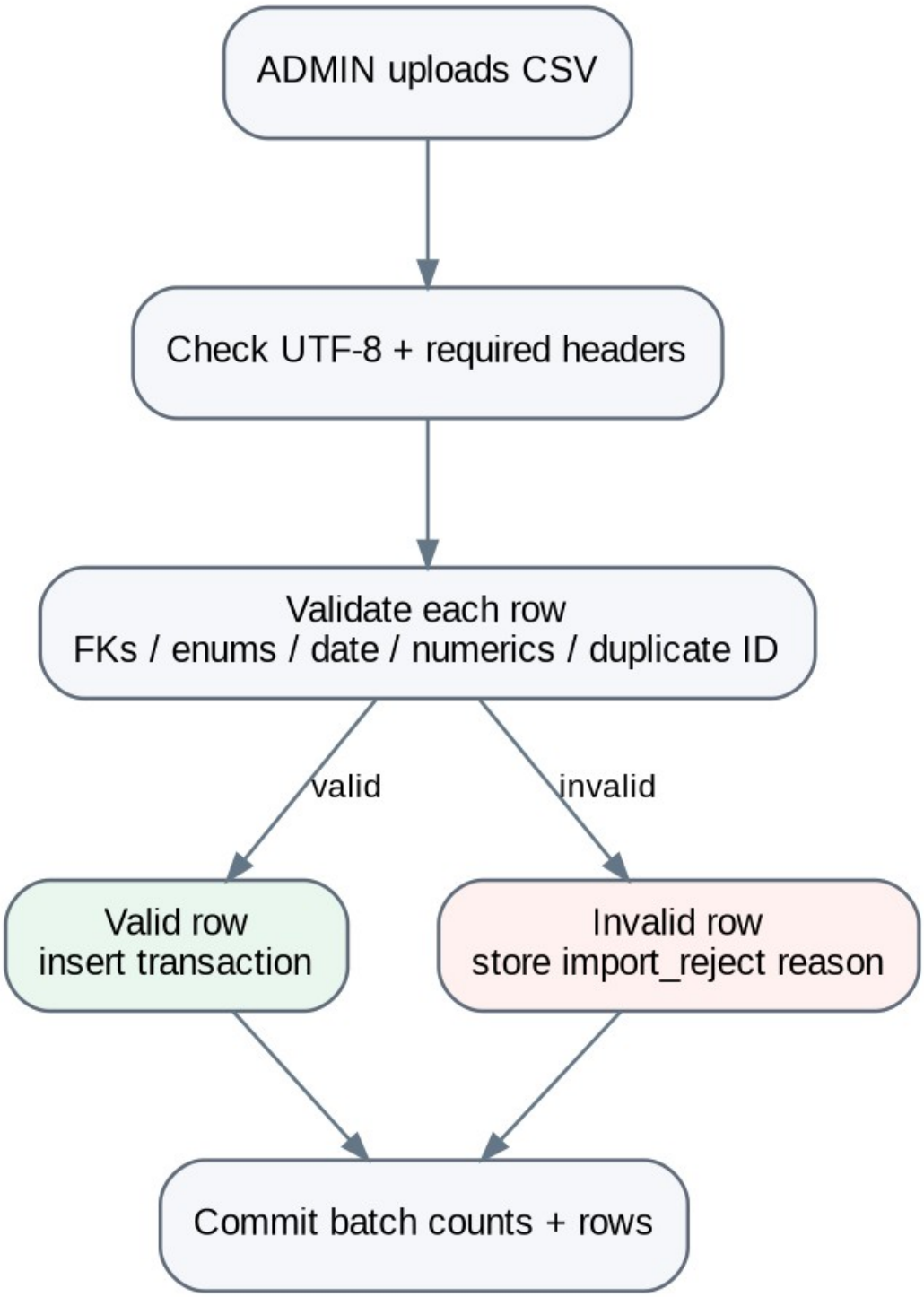


Figure 7. ADMIN transaction CSV import workflow.

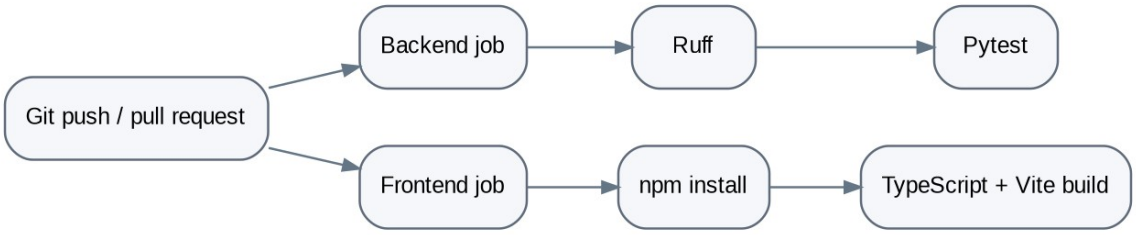


Figure 8. Configured CI workflow. A real successful run should be shown after the repository is pushed.

7. Local deployment topology

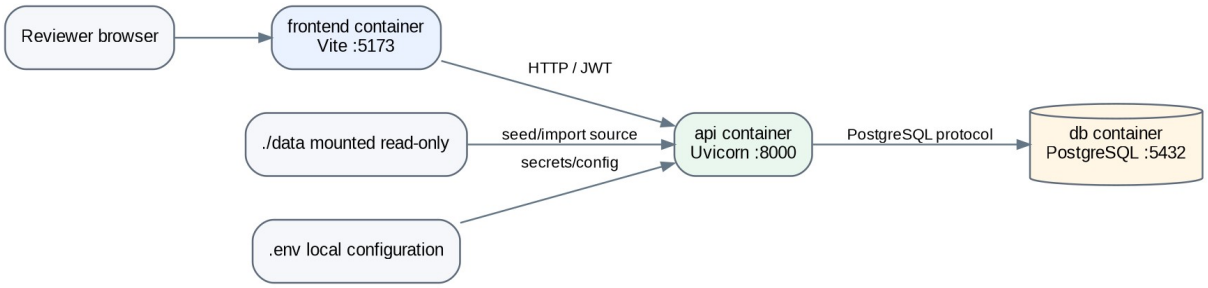


Figure 9. Local Docker Compose topology.

Service	Port	Startup responsibility
frontend	5173	Vite development server serving React UI.
api	8000	Alembic upgrade -> idempotent seed -> Uvicorn FastAPI server.
db	5432	PostgreSQL 16 with named local volume and health check.

The API waits for the database health check before startup. Raw datasets are mounted read-only into the API container. Local configuration is read from `.env`, while `.env.example` documents the required names without being a production secret store.

7.1 Production mapping

A production deployment would place the frontend/API behind a TLS ingress or reverse proxy, use a managed PostgreSQL service with backups and point-in-time recovery, inject secrets from a secret manager, remove development reload mode, use controlled migration jobs and collect central logs/metrics/traces. Larger imports would move to object storage and an asynchronous worker.

8. Security design

Concern	Current implementation	Production hardening
Authentication	Seeded local users; bcrypt password hashes; short-lived JWT.	Corporate OIDC/SSO and refresh/session lifecycle.
Authorization	Server-side VIEWER/ADMIN dependencies.	Central RBAC/ABAC policies and audit events.
Token storage	sessionStorage for assessment simplicity.	Secure HTTP-only SameSite cookie or server-managed session.
SQL injection	SQLAlchemy parameterization; bound parameters in targeted SQL.	Keep parameterized access; least-privilege DB role.
Input validation	Pydantic plus CSV row validation and DB constraints.	Schema contracts, malware/file scanning and rate limits.
Secrets	Local `.env`, git-ignored.	Secret manager + rotation.

CORS	Explicit localhost frontend origin.	Environment-specific allowlist.
PII/logging	No intentional password/token logging.	PII minimization, redaction and retention policy.

9. Reliability and operations

The `/health` endpoint checks API availability and database connectivity. Request middleware adds an `x-request-id`, measures duration and writes searchable completion/failure logs. Import batches retain imported/rejected counts and row-level rejection reasons so a bad data load can be explained rather than disappearing into a generic failure.

Area	Current approach	Next production step
Database availability	Single local PostgreSQL container.	Managed HA PostgreSQL, automated backups and PITR.
Connections	SQLAlchemy engine/session lifecycle.	Explicit pool sizing/timeouts based on measured concurrency.
Migrations	Alembic runs at local API startup.	Separate controlled deployment migration step; tested rollback/forward-fix.
Imports	Synchronous request transaction.	Queued/background worker, object storage and resumable batches.
Observability	Request IDs and application logs.	Central logs, metrics, tracing, alerts and SLOs.
Retries/timeouts	Normal local defaults.	Explicit outbound/DB timeouts and bounded retries where safe.

9.1 Query performance

Indexes target demonstrated query paths rather than every column. The transaction path uses `(account_id, trade_date)` for common account/date filtering; holdings use `(account_id, snapshot_date)`; customer search columns are indexed. `docs/sql-demo.sql` includes an `EXPLAIN (ANALYZE, BUFFERS)` query for the reviewer. The actual plan should be captured from the reviewer machine after the local database is populated.

10. Trade-offs, limitations and roadmap

Decision / limitation	Rationale / next step
Modular monolith, not microservices	Lower operational complexity and easier live tracing; split only when measured boundaries justify it.
Synchronous SQLAlchemy	Simple, explainable request path for the supplied scale; async is not automatically faster for this workload.
Seeded local auth	Meets local review need; replace with enterprise IdP/OIDC in production.
No frontend component tests	Backend validation/API path prioritized in timebox; add React Testing Library/E2E tests next.
No background import worker	Dataset is small; introduce queue/object storage for large files.
No TLS/reverse proxy locally	Plain localhost HTTP allowed by brief; production requires TLS and security headers.
CI evidence pending repository push	Workflow file is present; only a real successful GitHub Actions run counts as evidence.

The next production priorities would be enterprise authentication, central audit events, frontend/E2E tests, managed PostgreSQL backups, structured telemetry, dependency/security scanning, rate limiting and an asynchronous import pipeline.

Appendix A. Reviewer SQL demonstrations

The repository includes `docs/sql-demo.sql` with the six reviewer queries requested in the assignment: top customers by AUM, asset allocation, monthly BUY/SELL net cash flow, top instruments by holders, high-priority

underfunded goals and data-quality/import exceptions. It also contains the performance-plan query used with `EXPLAIN (ANALYZE, BUFFERS)`.

Appendix B. Reviewer demo path

Recommended live walkthrough: login as VIEWER -> customer search -> customer dashboard -> portfolio/asset allocation -> transaction filters -> goal create/edit -> logout -> login as ADMIN -> upload `data/transactions_import_demo.csv` -> show accepted/rejected counts -> open Swagger -> explain ERD and one SQL query -> show CI workflow and, after GitHub push, a successful Actions run.